VERA Project Summary

Professional engineering handover document for the VERA personal styling and wardrobe assistant application.

Repository: `veraFinal`

Primary frontend: `frontend/src`

Primary backend: `backend/main.py`

Database: MongoDB via PyMongo

AI provider: Google Gemini via `google-genai`

Storage: Local uploads served from `/uploads`

Executive Summary. VERA is a full-stack personal styling application that supports account creation, onboarding, wardrobe uploads, user preferences, outfit recommendations, AI chat, wishlist management, events, reviews, contact messages, and profile views. The main implementation uses a Vite React frontend and a FastAPI backend connected to MongoDB. A separate Express/Mongoose wardrobe server also exists under `server/`.

1. Project Overview

Purpose: Provide a digital wardrobe and styling assistant for authenticated users.

Files involved: `frontend/src/App.tsx`, `frontend/src/pages/*`, `frontend/src/lib/cloud.ts`, `backend/main.py`, `backend/routes/*`, `backend/database.py`, `server/index.js`.

How it works: React pages call API helpers in `cloud.ts` and auth helpers in `AuthContext.tsx`. FastAPI routes process requests and persist data in MongoDB collections declared in `database.py`.

Current status: MVP/prototype implemented with several complete workflows and some incomplete production features.

2. Purpose of the Application

Purpose: Help users upload wardrobe items, define style preferences, receive outfit suggestions, and consult an AI stylist.

Files involved: `Home.tsx`, `Wardrobe.tsx`, `Outfits.tsx`, `Chat.tsx`, `Preferences.tsx`, `backend/services/recommendation.py`, `backend/routes/chat.py`.

How it works: The user signs up, completes onboarding, uploads wardrobe items, then uses outfit generation and AI chat features driven by MongoDB wardrobe/preference data.

Current status: The core application purpose is represented in working code. AI chat requires Gemini configuration.

3. Technology Stack

Purpose: Define the runtime, framework, UI, database, and AI technologies used by the project.

Files involved: `frontend/package.json`, `backend/package.json`, `backend/requirements.txt`, `server/package.json`.

- Frontend: React 18, TypeScript, Vite, React Router, TanStack Query, Tailwind CSS, shadcn/Radix UI, lucide-react, sonner.
- Backend: FastAPI, Uvicorn, python-multipart, python-dotenv, google-genai, PyMongo usage in source.
- Alternate server: Express, Mongoose, Multer, CORS, dotenv.
- Database: MongoDB.
- AI: Google Gemini through `backend/services/gemini_service.py`.
- Weather: Open-Meteo calls from `frontend/src/lib/weather.ts`.

Current status: The stack is implemented but mixed, with FastAPI and Express backend code coexisting.

4. Frontend Architecture

Purpose: Provide routing, UI screens, auth state, API integration, and client-side feature workflows.

Files involved: `frontend/src/App.tsx`, `frontend/src/main.tsx`, `frontend/src/contexts/AuthContext.tsx`, `frontend/src/lib/cloud.ts`, `frontend/src/lib/storage.ts`, `frontend/src/pages/*`, `frontend/src/components/*`.

How it works: `App.tsx` defines public, protected, and onboarding-gated routes. `AuthProvider` stores the safe user object in localStorage. Pages use `cloud.ts` for backend calls.

Current status: Main pages are implemented. Legacy localStorage utilities remain in `storage.ts`.

5. Backend Architecture

Purpose: Provide REST APIs for auth, wardrobe, upload, outfits, chat, events, reviews, wishlist, preferences, and contact messages.

Files involved: `backend/main.py`, `backend/routes/*.py`, `backend/database.py`, `backend/schemas.py`, `backend/services/*`.

How it works: `main.py` creates the FastAPI app, configures CORS, mounts static uploads, and registers routers. Route modules use PyMongo collections from `database.py`.

Current status: FastAPI is the broadest backend implementation. `server/index.js` is an alternate wardrobe-only Express backend.

6. Database Architecture

Purpose: Store all persistent user and application data.

Files involved: `backend/database.py`, `backend/schemas.py`, `backend/db/wardrobe_store.py`, `backend/routes/*.py`.

How it works: MongoDB is configured from `MONGO_URI` and optional `MONGO_DB`. Collections are accessed directly through PyMongo objects.

Current status: MongoDB is the active database layer. No relational migrations exist in the inspected code.

7. MongoDB Collections

Purpose: Define the document stores used by the backend.

Files involved: `backend/database.py`, `backend/schemas.py`, `backend/db/wardrobe_store.py`, route modules.

- `users`: email, name, password_hash, onboarded, created_at, updated_at.
- `preferences`: email, style, lifestyle, location, restrictions, timestamps.
- `wardrobe`: filename, email, category, subcategory, color, pattern, style, seasons, occasions, fit, item_name, ai fields, worn_count.
- `outfits`: email, outfit_name, item_ids, reasoning, occasion, mood, season, scores, saved/worn/liked/disliked flags.
- `chat_history`: email, message, reply, created_at.
- `events`: email, date, event, location, created_at.
- `reviews`: email, rating, title, body, created_at.
- `wishlist`: email, item_ids, updated_at.
- `contact_messages`: email, name, contact_email, topic, body, created_at.

Current status: Collections and indexes are declared in `database.py`. Relationships are linked mostly by email and item ID strings.

8. Supabase Tables

Purpose: Requested area for Supabase schema documentation.

Files involved: No Supabase client, schema, package, or table implementation was found in the inspected files.

How it works: Supabase is not used by the current implemented application code.

Current status: No Supabase tables are implemented.

9. Storage Buckets

Purpose: Store wardrobe image assets.

Files involved: `backend/config.py`, `backend/main.py`, `backend/routes/upload.py`, `backend/uploads/`, `server/index.js`.

How it works: FastAPI writes files to `backend/uploads` and serves them at `/uploads`. The Express server has its own local `uploads` directory path.

Current status: Local filesystem storage is implemented. No Supabase bucket is implemented.

10. Authentication System

Purpose: Register users, log users in, maintain frontend auth state, and track onboarding completion.

Files involved: `AuthContext.tsx`, `Login.tsx`, `Signup.tsx`, `Onboarding.tsx`, `backend/routes/auth.py`, `backend/schemas.py`.

How it works: Backend stores salted PBKDF2-HMAC-SHA256 password hashes. Frontend stores the returned user object in localStorage and protects routes based on that state.

Current status: Auth is implemented, but it is not token/session based.

11. API Architecture

Purpose: Expose backend capabilities to the frontend through REST-style endpoints.

Files involved: `backend/main.py`, `backend/routes/*.py`, `frontend/src/lib/cloud.ts`, `frontend/src/contexts/AuthContext.tsx`.

How it works: Most routers are included with the `/api` prefix. Chat routes are included without the `/api` prefix.

Current status: API modules are implemented for the main application domains.

12. Wardrobe Management System

Purpose: Upload, list, filter, inspect, delete, and mark wardrobe items as worn.

Files involved: `Wardrobe.tsx`, `cloud.ts`, `backend/routes/upload.py`, `backend/routes/wardrobe.py`, `backend/db/wardrobe_store.py`.

How it works: Frontend sends multipart form data. Backend validates the file and metadata, writes the file locally, inserts a wardrobe document, and returns normalized item data.

Current status: Implemented with manual category/subcategory metadata. AI visual classification is not implemented.

13. Outfit Recommendation System

Purpose: Generate wardrobe-based outfits for occasion, mood, and season.

Files involved: `backend/services/recommendation.py`, `backend/routes/outfits.py`, `Outfits.tsx`, `cloud.ts`, `frontend/src/lib/recommendation.ts`, `frontend/src/lib/recommendations.ts`.

How it works: Backend groups items by category, builds combinations, scores them for color harmony, style compatibility, occasion, season, preferences, and feedback, then returns ranked outfits.

Current status: Backend recommendation logic is implemented. Frontend also contains separate recommendation code.

14. AI Features

Purpose: Provide AI stylist conversation and planned AI wardrobe metadata support.

Files involved: `backend/services/gemini_service.py`, `backend/routes/chat.py`, `Chat.tsx`, `cloud.ts`, `backend/config.py`.

How it works: Chat builds a wardrobe-aware Gemini prompt and stores replies in `chat_history`. If Gemini is not configured, the backend returns a configured fallback message.

Current status: AI chat is implemented. Upload image analysis is not implemented.

15. Upload & Image Processing Flow

Purpose: Accept wardrobe image uploads and create wardrobe records.

Files involved: `Wardrobe.tsx`, `cloud.ts`, `backend/routes/upload.py`, `backend/config.py`, `backend/uploads/`.

How it works: Backend checks required email/category, validates file extension and size, saves a UUID-named file, and inserts metadata into MongoDB.

Current status: Upload storage is implemented. Resizing, compression, and AI classification are not implemented in FastAPI.

16. Chat System

Purpose: Enable natural-language styling support.

Files involved: `Chat.tsx`, `cloud.ts`, `backend/routes/chat.py`, `backend/services/gemini_service.py`, `backend/database.py`.

How it works: Frontend loads history and sends messages. Backend sanitizes email, picks relevant wardrobe items, calls Gemini, stores message/reply records, and returns the response.

Current status: Implemented, with runtime behavior dependent on Gemini API configuration.

17. User Preference System

Purpose: Store style preferences for onboarding, profile display, and recommendations.

Files involved: `Onboarding.tsx`, `Preferences.tsx`, `Profile.tsx`, `cloud.ts`, `backend/routes/preferences.py`, `backend/schemas.py`.

How it works: Preferences are saved by email using `PUT /api/preferences/{email}` and read using `GET /api/preferences/{email}`.

Current status: Implemented and used across profile, home, and recommendation workflows.

18. Data Flow Architecture

Purpose: Describe movement of data through UI, API, storage, database, and AI services.

Files involved: `cloud.ts`, `AuthContext.tsx`, `backend/main.py`, `backend/routes/*`, `backend/database.py`, `backend/services/*`.

How it works: User action in React -> helper in `cloud.ts` or `AuthContext.tsx` -> FastAPI route -> MongoDB or local uploads -> normalized response -> frontend state update. Chat also calls Gemini. Weather is frontend-only through Open-Meteo.

Current status: Main flows are centralized around `cloud.ts`, with legacy localStorage utilities still present.

19. Current Implementation Status

Purpose: Summarize overall delivery maturity.

Files involved: Entire inspected repository source.

How it works: The project contains a complete MVP-style React/FastAPI/MongoDB vertical slice.

Current status: Suitable for final-year project demonstration and technical review. Additional hardening is needed for production.

20. Features Completed

Purpose: Document implemented capabilities.

Files involved: `frontend/src/pages/*`, `frontend/src/lib/cloud.ts`, `backend/routes/*`, `backend/services/*`.

- Signup, login, onboarding, and protected routing.
- Wardrobe upload, listing, deletion, and worn count update.
- Preferences create/update/read.
- Profile overview.
- Events, reviews, wishlist, and contact message flows.
- Gemini-backed chat path.
- Deterministic outfit recommendation engine.

Current status: These features are represented in working source code.

21. Features In Progress

Purpose: Document incomplete but visible feature areas.

Files involved: `Profile.tsx`, `cloud.ts`, `backend/routes/upload.py`, `backend/services/recommendation.py`.

- Profile avatar persistence is marked TODO in `Profile.tsx`.
- Collage generation rejects because it is not available in the local backend.
- AI wardrobe image analysis is not implemented despite `ai_analyzed` metadata.
- Recommendation logic exists in both frontend and backend.

Current status: These areas require implementation or consolidation.

22. Known Issues

Purpose: Provide handover context for behaviors requiring attention.

Files involved: `backend/routes/outfits.py`, `Signup.tsx`, `Profile.tsx`, `Wishlist.tsx`, `cloud.ts`, `backend/routes/upload.py`.

- `backend/routes/outfits.py`: route order can cause `/api/outfits/recommend/{email}` to be matched by `/{email}`.
- `backend/routes/outfits.py`: response mapping uses `reason` while schema defines `reasoning`.
- `Signup.tsx`: frontend allows 6-character passwords while backend requires 8.
- `Profile.tsx`: avatar add/remove does not persist to backend.
- `cloud.ts`: collage generation is explicitly unavailable.
- `Wishlist.tsx`: uses legacy wardrobe field names in rendering.
- `upload.py` and `Wardrobe.tsx`: backend and frontend upload size limits differ.

Current status: These are known handover items, not blockers for project demonstration.

23. Technical Strengths

Purpose: Highlight strong implementation choices.

Files involved: `database.py`, `schemas.py`, `auth.py`, `recommendation.py`, `cloud.ts`, `App.tsx`.

- Domain-based FastAPI route separation.
- Pydantic validation for backend request/response models.
- Salted PBKDF2 password hashing.
- MongoDB indexes for common lookup fields.
- Centralized frontend API layer in `cloud.ts`.
- Explainable deterministic outfit scoring.
- Clear protected-route and onboarding gate logic.

Current status: The project has a solid MVP engineering foundation.

24. Future Improvements

Purpose: Define practical next steps for production readiness.

Files involved: `auth.py`, `AuthContext.tsx`, `upload.py`, `gemini_service.py`, `recommendation.py`, `cloud.ts`, `server/index.js`.

- Add token/session-based authentication.
- Choose one backend path: FastAPI or Express.
- Consolidate duplicated frontend source under `frontend/src` and `backend/src`.
- Implement real AI image analysis for uploads.
- Persist profile avatars through backend APIs.
- Implement or remove collage generation UI.
- Unify recommendation logic between frontend and backend.
- Add automated tests for backend routes and frontend workflows.
- Clarify Supabase plans, since current source uses MongoDB and local filesystem storage.

Current status: Recommended improvements are clear and actionable for the next development phase.